# Lossless compression (II)
# Arithmetic coding, …

Karel Fliegel
(fliegek@fel.cvut.cz)

MultiMedia Technology Group

Czech Technical University in Prague
Faculty of Electrical Engineering
Department of Radioelectronics

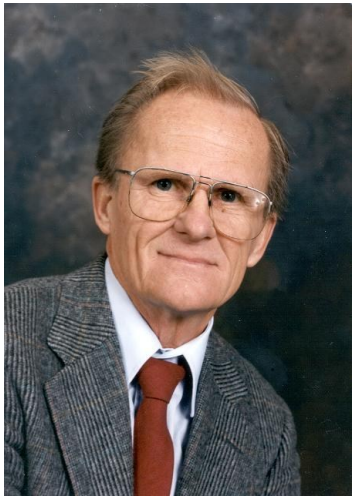
Technická 2
166 27 Prague 6
Czech Republic

6/3/2026

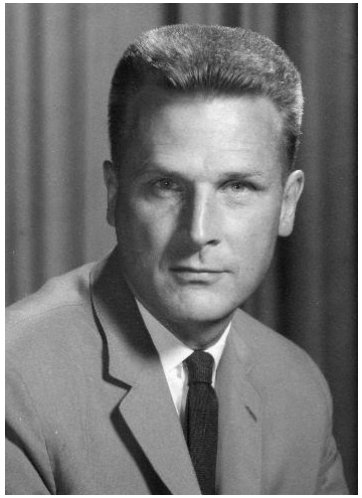

© Multimedia Technology Group

# Contents


- **Arithmetic coding**
  - **Introduction**
  - **Coding sequence**
  - **Generating binary code**
- **Adaptive arithmetic coding**
- **Binary arithmetic coding**
  - **QM** and **M coder**
- **Comparison** of **Huffman** and **arithmetic coding**
- **Applications**

**Jorma Rissanen**
(1932 – 2020)



**David Albert Huffman**
(1925 – 1999)

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.



© Multimedia Technology Group

# Arithmetic coding: Introduction

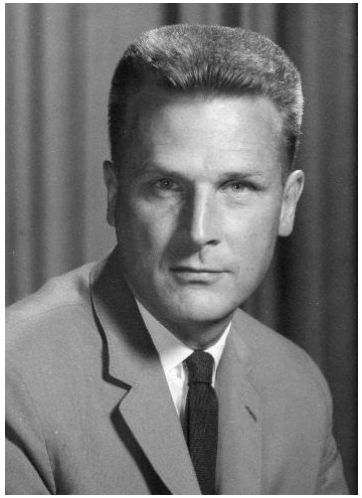


## Generating variable length codes

- **Huffman coding procedure** generates **optimum code**
  - **Average length** of **optimum code** depends on
    - **Size** of **alphabet** and **letter** (symbol) **probabilities**
  - **Optimum code** (Huffman code) for **source** $\mathcal{S}$ has **average code length** $\bar{l}$ **bounded** by **entropy** $H(\mathcal{S})$ and $H(\mathcal{S})+1$

$$H(\mathcal{S}) \leq \bar{l} < H(\mathcal{S}) + 1$$

  - More specifically **Huffman code rate** bounds
    - $H(\mathcal{S}) \leq \bar{l} < H(\mathcal{S}) + p_{max} + 0.086$ for $p_{max} < 0.5$
    - $H(\mathcal{S}) \leq \bar{l} < H(\mathcal{S}) + p_{max}$ for $p_{max} \geq 0.5$
    - For **most frequently occurring symbol** $p_{max}$
- **Arithmetic coding**
  - Especially **useful** when **dealing** with **small alphabets**
    - Typically **binary sources** and **highly skewed probabilities** (with large $p_{max}$) **alphabets**
    - **Useful** approach when **modeling** and **coding** aspects to be **separated**



**David Albert Huffman**
(1925 – 1999)

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Arithmetic coding: Introduction



## Huffman code procedure

### Example

- Designing **Huffman code** for **letters** from **alphabet** $\mathcal{A} = \{a_1, a_2, a_3\}$ with **probabilities** $P(a_1) = 0.95$, $P(a_2) = 0.02$ and $P(a_3) = 0.03$
- **Huffman code** for **three-letter alphabet**

| Letter | Codeword |
|---|---|
| $a_1$ | 0 |
| $a_2$ | 11 |
| $a_3$ | 10 |

  – Related **source entropy** $H = 0.335$ bit/symbol

  – Related **average Huffman code length**

$$\bar{l} = 0.95 \cdot 1 + 0.02 \cdot 2 + 0.03 \cdot 2 = 1.05 \text{ bit/symbol}$$

- Related **redundancy** (**entropy** and average **length difference**)

$$0.715 \text{ bit/symbol}$$

- **Redundancy** makes 213 % of **entropy**
- More than **twice bits needed** than **promised** by **entropy bound**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Arithmetic coding: Introduction



## Huffman code procedure

### Example

- Designing **Huffman code** for **letters** from **alphabet** $\mathcal{A} = \{a_1, a_2, a_3\}$ with **probabilities** $P(a_1) = 0.95$, $P(a_2) = 0.02$ and $P(a_3) = 0.03$
- **Huffman code** for **extended alphabet**

| Letter | Probability | Code |
|---|---|---|
| $a_1a_1$ | 0.9025 | 0 |
| $a_1a_2$ | 0.0190 | 111 |
| $a_1a_3$ | 0.0285 | 100 |
| $a_2a_1$ | 0.0190 | 1101 |
| $a_2a_2$ | 0.0004 | 110011 |
| $a_2a_3$ | 0.0006 | 110001 |
| $a_3a_1$ | 0.0285 | 101 |
| $a_3a_2$ | 0.0006 | 110010 |
| $a_3a_3$ | 0.0009 | 110000 |

  – Related **source entropy** $H = 0.335$ bit/symbol
  – Related **average extended Huffman code length** $\bar{l} = 0.611$ bit/symbol (**orig**)
  – Related **redundancy** (**entropy** and average **length difference**) 0.276 bit/symbol
  – **Redundancy** makes 82 % of entropy
- **Extended** codes → **impractical sizes** of code → **memory requirements** → **slow encoding/decoding**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Arithmetic coding: Introduction



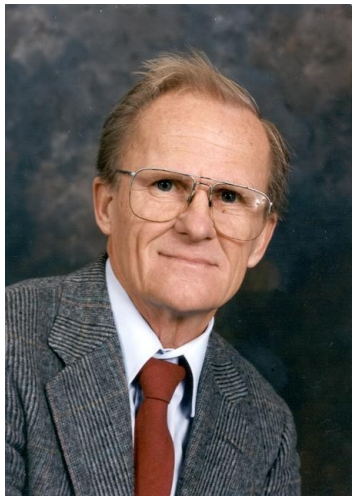
- **Observations** leading to **arithmetic coding**
  - More **efficient generating codewords** for **groups** or **sequences**
    - More **efficient** than **generating separate codewords** for **each symbol**
    - **Impractical** in **Huffman codes** for **long sequences** of **symbols**
      - **Finding Huffman codeword** for **sequence** of **length** $m$ → **codewords** for **all possible sequences** of **length** $m$ **needed**
      - **Exponential growth** in **codebook size**
      - **Procedure** of **assigning codewords** to **particular sequences without need** to **generate codes** for **all sequences** →
  - **Arithmetic coding**
    - **Identifier/Tag generated** for **sequence** to be **encoded**
      - **Tag corresponds** to **binary fraction** → **binary code** for **sequence**
    - **Can** be **divided** into **two phases**
      - (1) **Generating unique Tag identifier** for **given sequence** of **symbols**
      - (2) **Giving unique binary code** to **this identifier**
    - **No need** to **generate code** for **all sequences** of **length** $m$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Arithmetic coding: Coding sequence



- **Coding sequence** using **arithmetic coding**
  - **Tag** with **unique identifier** needed
    - **Tag** to **represent sequences** of **symbols** → **numbers** in **interval** $[0,1)$
      - **Infinite possibilities** to **assign unique Tag** to **each distinct sequence**
    - **Function** to **map sequences** into **unit interval** needed
      - **Cumulative distribution function** (**CDF**) maps **sequences** of **random variables** into **unit interval**
  - **Arithmetic code developed** based around **CDF**
    - **Historical notes**
      - **Claude Elwood Shannon** (1948) → **utilization** of **CDF** in **Shannon-Fano code**
      - **Peter Elias** (same **Robert Mario Fano class** at **MIT** as **David Albert Huffman**) → **recursive coding** technique **using CDF**
      - **Richard Pasco** (Stanford dissertation) and **Jorma Rissanen** (1976, Helsinki University of Technology) → **modern arithmetic coding principles** with **finite precision**



**Jorma Rissanen**
(1932 – 2020)

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Arithmetic coding: Coding sequence



## Coding sequence using arithmetic coding

### Establishing notation

- **Random variable** → **maps outcomes** (set of outcomes) to **real numbers**
  - **Example - coin tossing** can map e.g. **head** → 0, **tail** → 1
- **Mapping** of **source symbols** to **numbers** $X(a_i) = i$ **where** $a_i \in \mathcal{A}$ for **alphabet** of **discrete source** $\mathcal{A} = \{a_1, a_2, \dots, a_m\}$ and **random variable** $X$
- **Probability density function** (**PDF**) for **random variable**

$$P(X = i) = P(a_i)$$

- **Cumulative distribution function** (**CDF**) for **random variable**

$$F_X(i) = \sum_{k=1}^{i} P(X = k)$$

### Distinct value of $F_X(i)$

- For **each symbol** $a_i$ with **nonzero probability**
- This fact **used** in **development** of **arithmetic code**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Arithmetic coding: Coding sequence



## Generating tag in arithmetic coding

- **Reducing** size of **interval** where **Tag resides** with **symbols**
  - **Procedure** of **dividing unit interval** into **subintervals**
    - (**1**) **Division** into **subintervals** $[F_X(i-1), F_X(i)), i = 1, \dots, m \rightarrow$ **minimum CDF** value is **zero**, **maximum CDF** value is **one** → **associating** $[F_X(i-1), F_X(i))$ with **symbol** $a_i$
    - (**2**) First **symbol** $a_k$ then **Tag value** in **subinterval** $[F_X(k-1), F_X(k))$
    - (**3**) **Partitioning subinterval** in exactly **same proportions** $[F_X(k-1) + F_X(j-1) \times (F_X(k) - F_X(k-1)), F_X(k-1) + F_X(j) \times (F_X(k) - F_X(k-1)))$ **associating** with $a_j$
    - (**4**) Second **symbol** $a_j$ then **Tag value** in this **subinterval**
  - **Succeeding symbols** cause **Tag restricted** to **subinterval** partitioned in **same proportions**
- **Example**
  - Find **Tag** considering **three letter alphabet** $\mathcal{A} = \{a_1, a_2, a_3\}$ with **probabilities** $P(a_1) = 0.7$, $P(a_2) = 0.1$ and $P(a_3) = 0.2$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

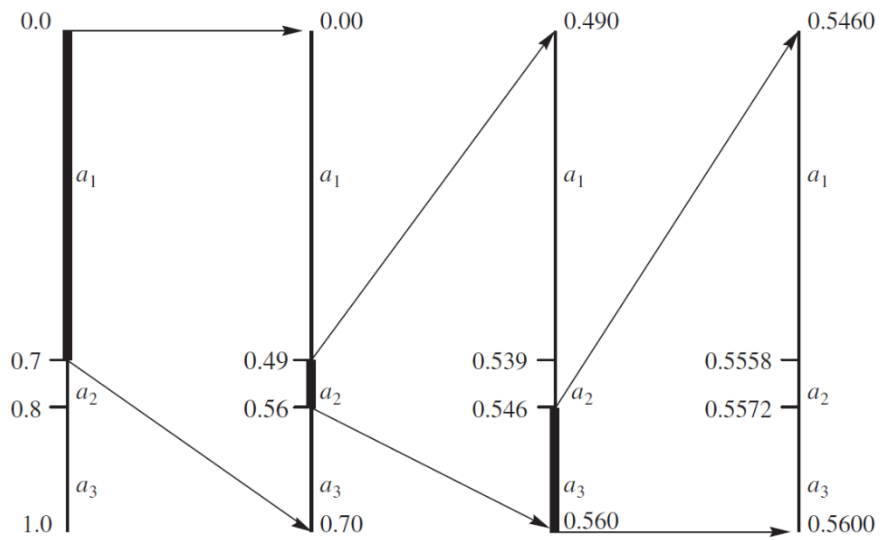
# Arithmetic coding: Coding sequence



## Generating tag in arithmetic coding

### Example

- Find **Tag** considering **three letter alphabet** $\mathcal{A} = \{a_1, a_2, a_3\}$ with **probabilities** $P(a_1) = 0.7$, $P(a_2) = 0.1$ and $P(a_3) = 0.2$
  - (**1**) Suppose **first symbol** $a_1$ → **Tag** in **subinterval** $[0.0, 0.7)$
  - (**2**) **Division** into **same portions** → $[0.0, 0.49)$, $[0.49, 0.56)$ and $[0.56, 0.70)$
  - (**3**) Suppose **second symbol** $a_2$ → **Tag** in **subinterval** $[0.49, 0.56)$
  - (**4**) **Division** into **same portions** → $[0.490, 0.539)$, $[0.539, 0.546)$ and $[0.546, 0.560)$
  - (**5**) Suppose **third symbol** $a_3$ → **Tag** in **subinterval** $[0.546, 0.560)$
  - (**6**) **Division** into **same portions** → $[0.5460, 0.5558)$, $[0.5558, 0.5572)$ and $[0.5572, 0.5600)$
  - …




Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Arithmetic coding: Coding sequence



- **Generating tag** in **arithmetic coding**
  - **Disjoint intervals** for **different symbol sequences**
  - **Any member** of **interval** can be **used** as **Tag**
    - **Lower limit** as **popular choice**
    - **Midpoint** as **popular alternative**
  - **Mapping** of **source** $\mathcal{A} = \{a_1, a_2, \dots, a_m\}$ **symbols** $\{a_i\}$ to **real numbers** $\{i\}$
  - **Defining tag** $\bar{T}_X(a_i)$

    $$\bar{T}_X(a_i) = \sum_{k=1}^{i-1} P(X = k) + \frac{1}{2} P(X = i) = F_X(i-1) + \frac{1}{2} P(X = i)$$

    - For **each** $a_i \rightarrow \bar{T}_X(a_i)$ **unique value** can be used as **Tag**
  - **Example**
    - Considering **simple dice** throwing with **fair dice** mapping **outcomes** into numbers $\{1, 2, \dots, 6\}$
      - **Probabilities** for **fair dice** $P(X = k) = 1/6$ for $k = 1, 2, \dots, 6$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Arithmetic coding: Coding sequence



## Generating tag in arithmetic coding

### Example

- Considering **simple dice** throwing with **fair dice** mapping **outcomes** into numbers $\{1, 2, \dots, 6\}$
  - **Probabilities** for **fair dice** $P(X = k) = 1/6$ for $k = 1, 2, \dots, 6$
  - **Tag** for $X = 2$ using $\bar{T}_X(a_i) = \sum_{k=1}^{i-1} P(X = k) + \frac{1}{2} P(X = i)$ **then**
    $\bar{T}_X(2) = P(X = 1) + \frac{1}{2} P(X = 2) = \frac{1}{6} + \frac{1}{12} = \frac{1}{4} = 0.25$
  - **Tag** for $X = 5$ using $\bar{T}_X(a_i) = \sum_{k=1}^{i-1} P(X = k) + \frac{1}{2} P(X = i)$ **then**
    $\bar{T}_X(5) = \sum_{k=1}^{4} P(X = k) + \frac{1}{2} P(X = 5) = 0.75$
  - Other **Tags**

| Outcome | Tag |
|---|---|
| 1 | $0.08\overline{33}$ |
| 3 | $0.41\overline{66}$ |
| 4 | $0.58\overline{33}$ |
| 6 | $0.91\overline{66}$ |

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Arithmetic coding: Coding sequence



## Generating tag in arithmetic coding

- **Tag** $\bar{T}_{\mathbf{X}}^{(m)}(\mathbf{x}_i)$ **extended** to **sequence** with **imposed order**

$$\bar{T}_{\mathbf{X}}^{(m)}(\mathbf{x}_i) = \sum_{\mathbf{y}<\mathbf{x}_i} P(\mathbf{y}) + \frac{1}{2} P(\mathbf{x_i})$$

  - Where $\mathbf{y} < \mathbf{x}$ **means** that $\mathbf{y}$ **precedes** $\mathbf{x}$ in ordering
  - Most common **lexicographical ordering** (dictionary order)

- **Example**
  - Considering **two rolls** of **dice** with **outcomes** in **dictionary order**
    - **Outcomes** for **fair dice** $11, 12, 13, \ldots, 66$
    - **Tag** for **sequence** $13$ **would** be $\bar{T}_X(13) = P(\mathbf{x} = 11) + P(\mathbf{x} = 12) + \frac{1}{2} P(\mathbf{x} = 13) = \frac{1}{36} + \frac{1}{36} + \frac{1}{2}\frac{1}{36} = \frac{5}{72}$
  - **Probability model** (probability of symbols) **has** to be **known**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Arithmetic coding: Coding sequence



## Generating tag in arithmetic coding

### Example

- Considering **simple dice** throwing with **fair dice** mapping **outcomes** into numbers $\{1, 2, \dots, 6\}$ and **sequence** 3 2 2 to be **encoded**
  - **Probabilities** for **fair dice** $P(X = k) = 1/6$ for $k = 1, 2, \dots, 6$
  - **Upper** $u^{(n)}$ and **lower** $l^{(n)}$ **limits** for **sequence** of **length** $n$
  - (**1**) Observation of **symbol** 3 → $u^{(n)} = F_X(3)$ and $l^{(n)} = F_X(2)$
  - (**2**) Observation of **symbol** 2 → **sequence** $\mathbf{x} = 32$ → $u^{(2)} = F_X^{(2)}(32)$ and $l^{(2)} = F_X^{(2)}(31)$
    - Computing values $F_X^{(2)}(32) = P(\mathbf{x} = 11) + P(\mathbf{x} = 12) + \cdots + P(\mathbf{x} = 32) = P(x_1 = 1) + P(x_1 = 2) + \cdots + P(\mathbf{x} = 31) + P(\mathbf{x} = 32) = F_X(2) + P(\mathbf{x} = 31) + P(\mathbf{x} = 32)$
    - For independent dice rolls $P(\mathbf{x} = 31) = P(x_1 = 3)P(x_2 = 1)$ and $P(\mathbf{x} = 32) = P(x_1 = 3)P(x_2 = 2)$ then $P(\mathbf{x} = 31) + P(\mathbf{x} = 32) = P(x_1 = 3)\big(P(x_2 = 1) + P(x_2 = 2)\big) = P(x_1 = 3)F_X(2)$
    - Noting $P(x_1 = 3) = F_X(3) - F_X(2)$ it can be written $P(\mathbf{x} = 31) + P(\mathbf{x} = 32) = \big(F_X(3) - F_X(2)\big)F_X(2)$ and $F_X^{(2)}(32) = F_X(2) + \big(F_X(3) - F_X(2)\big)F_X(2)$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Arithmetic coding: Coding sequence

## Generating tag in arithmetic coding

### Example

- Considering **simple dice** throwing with **fair dice** mapping **outcomes** into numbers $\{1, 2, \dots, 6\}$ and **sequence** $3\ 2\ 2$ to be **encoded**
  - (**2**) Observation of **symbol** $2 \rightarrow$ **sequence** $\mathbf{x} = 32 \rightarrow$
    $u^{(2)} = F_X^{(2)}(32)$ and $l^{(2)} = F_X^{(2)}(31)$
    - $F_X^{(2)}(32) = F_X(2) + \big(F_X(3) - F_X(2)\big)F_X(2)$ can be written as
      $u^{(2)} = l^{(1)} + \big(u^{(1)} - l^{(1)}\big)F_X(2)$ similarly
    - $F_X^{(2)}(31) = F_X(2) + \big(F_X(3) - F_X(2)\big)F_X(1)$ can be written as
      $l^{(2)} = l^{(1)} + \big(u^{(1)} - l^{(1)}\big)F_X(1)$
  - (**3**) Observation of **symbol** $2 \rightarrow$ **sequence** $\mathbf{x} = 322 \rightarrow$
    $u^{(3)} = F_X^{(3)}(322)$ and $l^{(3)} = F_X^{(3)}(321)$
    - $F_X^{(3)}(322) = F_X^{(2)}(31) + \left(F_X^{(2)}(32) - F_X^{(2)}(31)\right)F_X(2)$ can be written as
      $u^{(3)} = l^{(2)} + \big(u^{(2)} - l^{(2)}\big)F_X(2)$ similarly
    - $F_X^{(3)}(321) = F_X^{(2)}(31) + \left(F_X^{(2)}(32) - F_X^{(2)}(31)\right)F_X(1)$ can be written as
      $l^{(3)} = l^{(2)} + \big(u^{(2)} - l^{(2)}\big)F_X(1)$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.



© Multimedia Technology Group

# Arithmetic coding: Coding sequence



## Generating tag in arithmetic coding

- **For any sequence $\mathbf{x} = (x_1 x_2 \dots x_n)$ lower/upper** limits
  - $l^{(n)} = l^{(n-1)} + \left(u^{(n-1)} - l^{(n-1)}\right) F_X(x_n - 1)$
  - $u^{(n)} = l^{(n-1)} + \left(u^{(n-1)} - l^{(n-1)}\right) F_X(x_n)$
- **Midpoint** of this **interval** as **tag**
  - $\bar{T}_X(\mathbf{x}) = \frac{u^{(n)} + l^{(n)}}{2}$
  - **Tag** can be **computed** in **sequential fashion**
  - **Only** CDF needed (obtained from **probability model**)
- **Example**
  - Considering **three letter alphabet** $\mathcal{A} = \{a_1, a_2, a_3\}$ with **probabilities** $P(a_1) = 0.8$, $P(a_2) = 0.02$ and $P(a_3) = 0.18$ calculate **Tag** for **sequence $\mathbf{1\ 3\ 2\ 1}$**
    - (**1**) probability **model** $F_X(k) = 0$ for $k \leq 0$, $F_X(1) = 0.8$, $F_X(2) = 0.82$, $F_X(3) = 1$ and $F_X(k) = 1$ for $k > 3$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Arithmetic coding: Coding sequence

## Generating tag in arithmetic coding

### Example

- Considering **three letter alphabet** $\mathcal{A} = \{a_1, a_2, a_3\}$ with **probabilities** $P(a_1) = 0.8$, $P(a_2) = 0.02$ and $P(a_3) = 0.18$ calculate **Tag** for **sequence 1 3 2 1**
  - (**1**) probability **model** $F_X(k) = 0$ for $k \leq 0$, $F_X(1) = 0.8$, $F_X(2) = 0.82$, $F_X(3) = 1$ and $F_X(k) = 1$ for $k > 3$, **initialize** $u^{(0)} = 1$, $l^{(0)} = 0$
  - (**2**) **lower/upper** limits $l^{(n)} = l^{(n-1)} + \left(u^{(n-1)} - l^{(n-1)}\right)F_X(x_n - 1)$ and $u^{(n)} = l^{(n-1)} + \left(u^{(n-1)} - l^{(n-1)}\right)F_X(x_n)$ for first **sequence element 1**
    $l^{(1)} = 0 + (1 - 0)0 = 0$ and
    $u^{(1)} = 0 + (1 - 0)0.8 = 0.8 \rightarrow$ **Tag interval** $[0, 0.8)$
  - (**3**) **lower/upper** limits for second **sequence element 3**
    $l^{(2)} = 0 + (0.8 - 0)F_X(2) = 0.8 \cdot 0.82 = 0.656$ and
    $u^{(2)} = 0 + (0.8 - 0)F_X(3) = 0.8 \cdot 1.00 = 0.800 \rightarrow$ **Tag interval** $[0.656, 0.800)$
  - (**4**) **lower/upper** limits for third **sequence element 2**
    $l^{(3)} = 0.656 + (0.8 - 0.656)F_X(1) = 0.656 + 0.144 \cdot 0.8 = 0.7712$ and
    $u^{(3)} = 0.656 + (0.8 - 0.656)F_X(2) = 0.656 + 0.144 \cdot 0.82 = 0.77408 \rightarrow$ **Tag interval** $[0.7712, 0.77408)$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.



© Multimedia Technology Group

# Arithmetic coding: Coding sequence



## Generating tag in arithmetic coding

### Example

- Considering **three letter alphabet** $\mathcal{A} = \{a_1, a_2, a_3\}$ with **probabilities** $P(a_1) = 0.8$, $P(a_2) = 0.02$ and $P(a_3) = 0.18$ calculate **tag** for **sequence** $\mathbf{1\ 3\ 2\ 1}$
  - $l^{(n)} = l^{(n-1)} + \left(u^{(n-1)} - l^{(n-1)}\right)F_X(x_n - 1)$ and $u^{(n)} = l^{(n-1)} + \left(u^{(n-1)} - l^{(n-1)}\right)F_X(x_n)$
  - **(5) lower/upper** limits for fourth (last) **sequence element** $\mathbf{1}$ $l^{(4)} = 0.7712 + (0.77408 - 0.7712)F_X(0) = 0.7712 + 0.00288 \cdot 0.0 = 0.7712$ and $u^{(4)} = 0.7712 + (0.77408 - 0.7712)F_X(1) = 0.7712 + 0.00288 \cdot 0.8 = 0.773504$ → **Tag interval** $[0.7712, 0.773504)$
  - **(6) Tag** for **sequence** $\mathbf{1\ 3\ 2\ 1}$ can be **generated** as **midpoint**

$$\bar{T}_X(1\ 3\ 2\ 1) = \frac{0.7712 + 0.773504}{2} = 0.772352$$

- **Succeeding interval** contained in **preceding** → **used** in **deciphering**
  - Intervals get **smaller** and **smaller** → **higher precision** for **longer sequences**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Arithmetic coding: Coding sequence



## Deciphering tag in arithmetic coding

- **Tag** to be **deciphered** with **minimal computational cost**
- **Example**
  - Considering **three letter alphabet** $\mathcal{A} = \{a_1, a_2, a_3\}$ with **probabilities** $P(a_1) = 0.8$, $P(a_2) = 0.02$ and $P(a_3) = 0.18$ and **Tag** for **sequence** $\bar{T}_X = 0.772352$
    - **Upper/lower** limits ($u^{(k)}$ and $l^{(k)}$) **contain Tag value** for each $k$
    - **(1)** probability **model** $F_X(k) = 0$ for $k \leq 0$, $F_X(1) = 0.8$, $F_X(2) = 0.82$, $F_X(3) = 1$ and $F_X(k) = 1$ for $k > 3$
    - **(2) starting** with $l^{(0)} = 0$ and $u^{(0)} = 1$ **lower/upper** limits
    - **(3)** decoding **first sequence element** $x_1$
      $l^{(n)} = l^{(n-1)} + \left(u^{(n-1)} - l^{(n-1)}\right)F_X(x_n - 1)$ and
      $u^{(n)} = l^{(n-1)} + \left(u^{(n-1)} - l^{(n-1)}\right)F_X(x_n)$
      $l^{(1)} = 0 + (1 - 0)F_X(x_1 - 1) = F_X(x_1 - 1)$ and
      $u^{(1)} = 0 + (1 - 0)F_X(x_1) = F_X(x_1)$ → **Tag interval** $[F_X(x_1 - 1), F_X(x_1))$ → **choosing** $x_1$ for tag $\bar{T}_X = 0.772352$ **contained** in **interval** → $x_1 = \mathbf{1}$ → $\bar{T}_X = 0.772352 \in [0, 0.8)$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Arithmetic coding: Coding sequence



## Deciphering tag in arithmetic coding

- **Tag** to be **deciphered** with **minimal computational cost**
- **Example**
  - Considering **three letter alphabet** $\mathcal{A} = \{a_1, a_2, a_3\}$ with **probabilities** $P(a_1) = 0.8$, $P(a_2) = 0.02$ and $P(a_3) = 0.18$ and **Tag** for **sequence** $\bar{T}_X = 0.772352$
    - probability **model** $F_X(k) = 0$ for $k \leq 0$, $F_X(1) = 0.8$, $F_X(2) = 0.82$, $F_X(3) = 1$ and $F_X(k) = 1$ for $k > 0$
    - $x_1 = \mathbf{1} \rightarrow \bar{T}_X = 0.772352 \in [0, 0.8)$
    - **(4)** decoding **second sequence element** $x_2$
      $l^{(n)} = l^{(n-1)} + \left(u^{(n-1)} - l^{(n-1)}\right) F_X(x_n - 1)$ and
      $u^{(n)} = l^{(n-1)} + \left(u^{(n-1)} - l^{(n-1)}\right) F_X(x_n)$
      $l^{(2)} = 0 + (0.8 - 0) F_X(x_2 - 1) = 0.8\, F_X(x_2 - 1)$ and
      $u^{(2)} = 0 + (0.8 - 0) F_X(x_2) = 0.8\, F_X(x_2) \rightarrow$ **Tag interval** $[0.8\, F_X(x_2 - 1), 0.8\, F_X(x_2)) \rightarrow$ **choosing** $x_2$ for Tag $\bar{T}_X = 0.772352$ **contained** in **interval** $\rightarrow$ $x_2 = \mathbf{3} \rightarrow \bar{T}_X = 0.772352 \in [0.656, 0.8)$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Arithmetic coding: Coding sequence



## Deciphering tag in arithmetic coding

- **Tag** to be **deciphered** with **minimal computational cost**
- **Example**
  - Considering **three letter alphabet** $\mathcal{A} = \{a_1, a_2, a_3\}$ with **probabilities** $P(a_1) = 0.8$, $P(a_2) = 0.02$ and $P(a_3) = 0.18$ and **Tag** for **sequence** $\bar{T}_X = 0.772352$
    - probability **model** $F_X(k) = 0$ for $k \leq 0$, $F_X(1) = 0.8$, $F_X(2) = 0.82$, $F_X(3) = 1$ and $F_X(k) = 1$ for $k > 0$
    - $x_2 = \mathbf{3} \rightarrow \bar{T}_X = 0.772352 \in [0.656, 0.8)$
    - **(5)** decoding **third sequence element** $x_3$
      $l^{(n)} = l^{(n-1)} + \left(u^{(n-1)} - l^{(n-1)}\right) F_X(x_n - 1)$ and
      $u^{(n)} = l^{(n-1)} + \left(u^{(n-1)} - l^{(n-1)}\right) F_X(x_n)$
      $l^{(3)} = 0.656 + (0.8 - 0.656) F_X(x_3 - 1) = 0.656 + 0.144\, F_X(x_3 - 1)$ and
      $u^{(3)} = 0.656 + (0.8 - 0.656) F_X(x_3) = 0.656 + 0.144\, F_X(x_3) \rightarrow$ **Tag interval** $[0.656 + 0.144\, F_X(x_3 - 1), 0.656 + 0.144\, F_X(x_3)) \rightarrow$ **choosing** $x_3$ for Tag $\bar{T}_X = 0.772352$ **contained** in **interval** $\rightarrow x_3 = \mathbf{2} \rightarrow \bar{T}_X = 0.772352 \in [0.7712, 0.77408)$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Arithmetic coding: Coding sequence



## Deciphering tag in arithmetic coding

- **Tag** to be **deciphered** with **minimal computational cost**
- **Example**
  - Considering **three letter alphabet** $\mathcal{A} = \{a_1, a_2, a_3\}$ with **probabilities** $P(a_1) = 0.8$, $P(a_2) = 0.02$ and $P(a_3) = 0.18$ and **Tag** for **sequence** $\bar{T}_X = 0.772352$
    - probability **model** $F_X(k) = 0$ for $k \leq 0$, $F_X(1) = 0.8$, $F_X(2) = 0.82$, $F_X(3) = 1$ and $F_X(k) = 1$ for $k > 0$
    - $x_3 = \mathbf{2} \rightarrow \bar{T}_X = 0.772352 \in [0.7712, 0.77408)$
    - **(6)** decoding **fourth sequence element** $x_3$
      $l^{(n)} = l^{(n-1)} + \left(u^{(n-1)} - l^{(n-1)}\right) F_X(x_n - 1)$ and
      $u^{(n)} = l^{(n-1)} + \left(u^{(n-1)} - l^{(n-1)}\right) F_X(x_n)$
      $l^{(4)} = 0.7712 + (0.77408 - 0.7712) F_X(x_4 - 1) = 0.7712 + 0.00288\, F_X(x_4 - 1)$ and
      $u^{(4)} = 0.7712 + (0.77408 - 0.7712) F_X(x_4) = 0.7712 + 0.00288\, F_X(x_4) \rightarrow$
      **Tag interval** $[0.7712 + 0.00288\, F_X(x_4 - 1), 0.7712 + 0.00288\, F_X(x_4)) \rightarrow$
      **choosing** $x_4$ for tag $\bar{T}_X = 0.772352$ **contained** in **interval** $\rightarrow x_4 = \mathbf{1}$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Arithmetic coding: Coding sequence

- **Deciphering tag** in **arithmetic coding**
  - **Tag** to be **deciphered** with **minimal computational cost**
  - **Example**
    - Considering **three letter alphabet** $\mathcal{A} = \{a_1, a_2, a_3\}$ with **probabilities** $P(a_1) = 0.8$, $P(a_2) = 0.02$ and $P(a_3) = 0.18$ and **Tag** for **sequence** $\bar{T}_X = 0.772352$
      - probability **model** $F_X(k) = 0$ for $k \leq 0$, $F_X(1) = 0.8$, $F_X(2) = 0.82$, $F_X(3) = 1$ and $F_X(k) = 1$ for $k > 0$
      - **Deciphered** sequence **1 3 2 1**
    - **Length** of **sequence known** → to **know where** to **stop deciphering**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.



© Multimedia Technology Group

# Arithmetic coding: Coding sequence



## Deciphering tag in arithmetic coding

- **Tag** to be **deciphered** with **minimal computational cost**

- **General algorithm** for **Tag deciphering**
  - (**1**) **Initialize** $l^{(0)} = 0, u^{(0)} = 1$
  - (**2**) For **each** $k$ **find** $t^* = \frac{\mathrm{Tag} - l^{(k-1)}}{u^{(k-1)} - l^{(k-1)}}$
  - (**3**) Find **value** of $x_k$ **fulfilling** $F_X(x_k - 1) \leq t^* < F_X(x_k)$
  - (**4**) **Update** $l^{(k)}$ and $u^{(k)}$
  - (**5**) **Repeat steps** (**2-4**) until **entire sequence decoded**
- **Two ways** of **knowing** that **entire sequence decoded**
  - **Length might** be **known** for **decoder** → **deciphering stopped** after **all symbols decoded**
  - **Special end-of-transmission symbol** → **receiving** this **symbol stops decoding**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Arithmetic coding: Generating binary code



## Arithmetic coding and binary code

- **Binary code** to **represent sequence** $x$
  - **Unique** and **efficient representation needed**
  - **Tag forms unique representation** for **encoded sequence**
  - **Unique** and **efficient binary representation** of **Tag needed**
    - **No restrictions** on **Tag values** → infinitely **long binary representation**
    - **Efficient Tag representation** → **truncation** of **binary representation**
- **Uniqueness** and **efficiency**
  - **Tag** takes **value** $\bar{T}_X(x) \in [0,1)$ in **interval** $[0,1)$
    - **Truncating Tag binary code** to $l(x) = \left\lceil \log_2 \frac{1}{P(x)} \right\rceil + 1$ **bits**
    - Using **binary representation** of **numbers**
      $.b_1 b_2 \ldots b_k = b_1 2^{-1} + b_2 2^{-2} + \cdots + b_k 2^{-k}$ **thus** $.101 = 2^{-1} + 2^{-3}$
- **Example**
  - Consider **four letter alphabet** $\mathcal{A} = \{a_1, a_2, a_3, a_4\}$ with **probabilities** $P(a_1) = 1/2$, $P(a_2) = 1/4$, $P(a_3) = 1/8$ and $P(a_4) = 1/8$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Arithmetic coding: Generating binary code



## Arithmetic coding and binary code

### Example and arithmetic code properties

- Consider **four letter alphabet** $\mathcal{A} = \{a_1, a_2, a_3, a_4\}$ with **probabilities** $P(a_1) = 1/2$, $P(a_2) = 1/4$, $P(a_3) = 1/8$ and $P(a_4) = 1/8$
  - **Binary code** for **four-letter alphabet**
  - Obtaining **tag** using above **described procedure**

| Symbol | $F_X$ | $\bar{T}_X$ | In Binary | $\lceil \log \frac{1}{P(x)} \rceil + 1$ | Code |
|---|---|---|---|---|---|
| 1 | .500 | .2500 | .0100 | 2 | 01 |
| 2 | .750 | .6250 | .1010 | 3 | 101 |
| 3 | .875 | .8125 | .1101 | 4 | 1101 |
| 4 | 1.000 | .9375 | .1111 | 4 | 1111 |

- **Truncated representation** of $\bar{T}_X(x) \rightarrow \lfloor \bar{T}_X(x) \rfloor_{l(\mathbf{x})}$ **into** $l(\mathbf{x})$ **bits**
  - It **can be shown** → **truncated representation** $\lfloor \bar{T}_X(x) \rfloor_{l(\mathbf{x})}$ is **unique representation** of $\bar{T}_X(x)$
- **Assigned code** must be **uniquely decodable**
  - It **can be shown** → **obtained code** for $\lfloor \bar{T}_X(x) \rfloor_{l(\mathbf{x})}$ **in** $l(\mathbf{x})$ **bits** is **prefix code** → **no codeword prefix** of **another codeword** → **arithmetic code** is **uniquely decodable**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Arithmetic coding: Generating binary code



## Arithmetic coding and binary code

### Arithmetic code properties

- **Truncated representation** of $\bar{T}_X(x) \rightarrow \lfloor \bar{T}_X(x) \rfloor_{l(\mathbf{x})}$ **into** $l(\mathbf{x})$ **bits** → **unique representation** of $\bar{T}_X(x)$
- **Assigned code** for $\lfloor \bar{T}_X(x) \rfloor_{l(\mathbf{x})}$ **in** $l(\mathbf{x})$ **bits** is **uniquely decodable**
- **Average length** of **arithmetic code** $\bar{l}_A$ for **sequence** of **length** $m$
  - It **can be shown** → **average code length** $\bar{l}_A$ → $H(X) \leq \bar{l}_A < H(X) + \frac{2}{m}$
  - **Increasing sequence length** $m$ → $\bar{l}_A$ as **close** to **entropy** $H(X)$ as **desired**

### Arithmetic coding implementation

- **Recursive algorithm** for **interval boundaries** containing **Tag**
  - **Lower** $l^{(n)}$ and **upper** $u^{(n)}$ **limits** for **sequence** of **length** $n$
    $l^{(n)} = l^{(n-1)} + \left(u^{(n-1)} - l^{(n-1)}\right) F_X(x_n - 1)$ and
    $u^{(n)} = l^{(n-1)} + \left(u^{(n-1)} - l^{(n-1)}\right) F_X(x_n)$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Arithmetic coding: Generating binary code

## Arithmetic coding and binary code

### Arithmetic coding implementation

- **Recursive algorithm** for **interval boundaries** containing **Tag**
  - **Lower** $l^{(n)}$ and **upper** $u^{(n)}$ **limits** for **sequence** of **length** $n$
    $l^{(n)} = l^{(n-1)} + \left(u^{(n-1)} - l^{(n-1)}\right) F_X(x_n - 1)$ and
    $u^{(n)} = l^{(n-1)} + \left(u^{(n-1)} - l^{(n-1)}\right) F_X(x_n)$
  - For $n$ **getting larger** → values $l^{(n)}$ and $u^{(n)}$ come **closer** and **closer**
  - For $n$ **getting larger** → **increased** machine **precision needed**
- **Systems** with **finite precision** → **rescaling** of **interval**
  - **Information** to be **transmitted** has to be **preserved**
  - **Encoding** to be done **incrementally** → **transmitting portions** of **code** rather than **waiting** for **whole sequence** to transmit **first bit**
- With **interval becoming narrower** three solutions
  - (**1**) **Interval** entirely **confined** to **lower half** $[0, 0.5)$
  - (**2**) **Interval** entirely **confined** to **upper half** $[0.5, 1.0)$
  - (**3**) **Interval** straddles **midpoint** of **unit interval**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.



© Multimedia Technology Group

# Arithmetic coding: Generating binary code



## Arithmetic coding and binary code

### Arithmetic coding implementation

- With **interval becoming narrower** three solutions
  - (**1**) **Interval** entirely **confined** to **lower half** $[0, 0.5)$
    - **Most significant bit** (**MSB**) for **all numbers** in $[0, 0.5)$ is $0$
  - (**2**) **Interval** entirely **confined** to **upper half** $[0.5, 1.0)$
    - **Most significant bit** (**MSB**) for **all numbers** in $[0.5, 1.0)$ is $1$
  - For **interval restricted** to **upper/lower** part → **MSB** of Tag is **fully determined**
  - **Without waiting** for **whole sequence** → **upper/lower indicated** to **decoder** in **MSB**
- **Mapping half interval** containing **Tag** to **full interval** $[0, 1)$
  - (**1**) $E_1: [0, 0.5) \rightarrow [0, 1) \qquad E_1(x) = 2x$
  - (**2**) $E_2: [0.5, 1) \rightarrow [0, 1) \qquad E_2(x) = 2(x - 0.5)$
  - **After mapping** → **information** about **MSB** is **lost**
  - **After mapping loosing MSB** → **no problem** since **MSB already** sent to **decoder**
  - Allows for **generating bits** of **tag without waiting** for **entire sequence** → **incremental encoding**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Arithmetic coding: Generating binary code



## Arithmetic coding and binary code

### Example

- Considering **three letter alphabet** $\mathcal{A} = \{a_1, a_2, a_3\}$ with **probabilities** $P(a_1) = 0.8$, $P(a_2) = 0.02$ and $P(a_3) = 0.18$ calculate **Tag** for **sequence $\mathbf{1\ 3\ 2\ 1 \rightarrow 1}$**
  - **(1)** probability **model** $F_X(k) = 0$ for $k \leq 0$, $F_X(1) = 0.8$, $F_X(2) = 0.82$, $F_X(3) = 1$ and $F_X(k) = 1$ for $k > 0$
  - **(2) lower/upper** limits $l^{(n)} = l^{(n-1)} + \left(u^{(n-1)} - l^{(n-1)}\right)F_X(x_n - 1)$ and $u^{(n)} = l^{(n-1)} + \left(u^{(n-1)} - l^{(n-1)}\right)F_X(x_n)$ for first **sequence element 1**
    $l^{(1)} = 0 + (1 - 0)0 = 0$ and
    $u^{(1)} = 0 + (1 - 0)0.8 = 0.8 \rightarrow$ **Tag interval** $[0, 0.8) \rightarrow$ **not confined**
  - **(3) lower/upper** limits for second **sequence element 3**
    $l^{(2)} = 0 + (0.8 - 0)F_X(2) = 0.8 \cdot 0.82 = 0.656$ and
    $u^{(2)} = 0 + (0.8 - 0)F_X(3) = 0.8 \cdot 1.00 = 0.800 \rightarrow$ **Tag interval** $[0.656, 0.800)$
    $\rightarrow$ **confined upper half** $\rightarrow$ **sending binary 1** and **rescale**
  - **(4) rescaling** $E_2: [0.5, 1) \rightarrow [0, 1)$ $\quad E_2(x) = 2(x - 0.5)$
    $l^{(2)} = 2 \cdot (0.656 - 0.5) = 0.312$ and $u^{(2)} = 2 \cdot (0.8 - 0.5) = 0.6 \rightarrow$
    **Tag interval** $[0.312, 0.600)$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Arithmetic coding: Generating binary code

## Arithmetic coding and binary code

### Example

- Considering **three letter alphabet** $\mathcal{A} = \{a_1, a_2, a_3\}$ with **probabilities** $P(a_1) = 0.8$, $P(a_2) = 0.02$ and $P(a_3) = 0.18$ calculate **Tag** for **sequence** $\mathbf{1\ 3\ 2\ 1 \rightarrow 1\ 1\ 0\ 0}$
  - **(5) lower/upper** limits for third **sequence element 2**
    $l^{(3)} = 0.312 + (0.6 - 0.312)F_X(1) = 0.312 + 0.288 \cdot 0.8 = 0.5424$ and $u^{(3)} = 0.312 + (0.8 - 0.312)F_X(2) = 0.312 + 0.288 \cdot 0.82 = 0.54816$ → **Tag interval** $[0.5424, 0.54816)$ → **confined upper half** → **sending binary 1** and **rescale**
  - **(6) rescaling** $E_2$: $[0.5, 1) \rightarrow [0, 1)$ $\quad E_2(x) = 2(x - 0.5)$
    $l^{(3)} = 2 \cdot (0.54816 - 0.5424) = 0.0848$ and $u^{(3)} = 2 \cdot (0.54816 - 0.5) = 0.09632$ → **Tag interval** $[0.0848, 0.09632)$ → **confined lower half** → **sending binary 0** and **rescale**
  - **(7) rescaling** $E_1$: $[0, 0.5) \rightarrow [0, 1)$ $\quad E_1(x) = 2x$
    $l^{(3)} = 2 \cdot (0.0848) = 0.1696$ and $u^{(3)} = 2 \cdot (0.09632) = 0.19264$ → **Tag interval** $[0.1696, 0.19264)$ → **confined lower half** → **sending binary 0** and **rescale**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.



© Multimedia Technology Group

# Arithmetic coding: Generating binary code

## Arithmetic coding and binary code

### Example

- Considering **three letter alphabet** $\mathcal{A} = \{a_1, a_2, a_3\}$ with **probabilities** $P(a_1) = 0.8$, $P(a_2) = 0.02$ and $P(a_3) = 0.18$ calculate **Tag** for **sequence** $\mathbf{1\ 3\ 2\ 1 \rightarrow 1\ 1\ 0\ 0\ 0\ 1}$
  - **(7) rescaling** $E_1: [0, 0.5) \rightarrow [0, 1)$ $\quad E_1(x) = 2x$
    $l^{(3)} = 2 \cdot (0.0848) = 0.1696$ and $u^{(3)} = 2 \cdot (0.09632) = 0.19264$ → **Tag interval** $[0.1696, 0.19264)$ → **confined lower half** → **sending binary 0** and **rescale**
  - **(8) rescaling** $E_1: [0, 0.5) \rightarrow [0, 1)$ $\quad E_1(x) = 2x$
    $l^{(3)} = 2 \cdot (0.1696) = 0.3392$ and $u^{(3)} = 2 \cdot (0.19264) = 0.38528$ → **Tag interval** $[0.3392, 0.38528)$ → **confined lower half** → **sending binary 0** and **rescale**
  - **(9) rescaling** $E_1: [0, 0.5) \rightarrow [0, 1)$ $\quad E_1(x) = 2x$
    $l^{(3)} = 2 \cdot (0.3392) = 0.6784$ and $u^{(3)} = 2 \cdot (0.38528) = 0.77056$ → **Tag interval** $[0.6784, 0.77056)$ → **confined upper half** → **sending binary 1** and **rescale**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.



© Multimedia Technology Group

# Arithmetic coding: Generating binary code



## Arithmetic coding and binary code

### Example

- Considering **three letter alphabet** $\mathcal{A} = \{a_1, a_2, a_3\}$ with **probabilities** $P(a_1) = 0.8$, $P(a_2) = 0.02$ and $P(a_3) = 0.18$ calculate **Tag** for **sequence** $\mathbf{1\ 3\ 2\ 1 \rightarrow 1\ 1\ 0\ 0\ 0\ 1}$
  - **(9) rescaling** $E_1: [0, 0.5) \rightarrow [0, 1)$ $\quad E_1(x) = 2x$
    $l^{(3)} = 2 \cdot (0.3392) = 0.6784$ and $u^{(3)} = 2 \cdot (0.38528) = 0.77056$ → **Tag interval** $[0.6784, 0.77056)$ → **confined upper half** → **sending binary 1** and **rescale**
  - **(10) rescaling** $E_2: [0.5, 1) \rightarrow [0, 1)$ $\quad E_2(x) = 2(x - 0.5)$
    $l^{(3)} = 2 \cdot (0.6784 - 0.5) = 0.3568$ and $u^{(3)} = 2 \cdot (0.77056 - 0.5) = 0.54112$ → **Tag interval** $[0.3568, 0.54112)$ → **not confined**
  - **(11) lower/upper** limits for fourth **sequence element 1**
    $l^{(4)} = 0.3568 + (0.54112 - 0.3568)F_X(0) = 0.3568 + 0.18422 \cdot 0.0 = 0.3568$ and
    $u^{(4)} = 0.3568 + (0.54112 - 0.3568)F_X(1) = 0.3568 + 0.18422 \cdot 0.8 = 0.504256$ → **Tag interval** $[0.3568, 0.504256)$ → **not confined**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Arithmetic coding: Generating binary code



## Arithmetic coding and binary code

### Example

- Considering **three letter alphabet** $\mathcal{A} = \{a_1, a_2, a_3\}$ with **probabilities** $P(a_1) = 0.8$, $P(a_2) = 0.02$ and $P(a_3) = 0.18$ calculate **Tag** for **sequence** $\mathbf{1\ 3\ 2\ 1} \rightarrow \mathbf{1\ 1\ 0\ 0\ 0\ 1\ 1}$
  - (**11**) **lower/upper** limits for fourth **sequence element 1**
    $l^{(4)} = 0.3568 + (0.54112 - 0.3568)F_X(0) = 0.3568 + 0.18422 \cdot 0.0 = 0.3568$ and
    $u^{(4)} = 0.3568 + (0.54112 - 0.3568)F_X(1) = 0.3568 + 0.18422 \cdot 0.8 = 0.504256$ → **Tag interval** $[0.3568, 0.504256)$ → **not confined**
  - (**12**) **Stop encoding** → **informing receiver** of **final Tag** value → **convenient value** 0.5 → **binary representation** .1 **followed** by **required number** of 0s
  - (**13**) **Binary sequence** generated $\mathbf{1\ 3\ 2\ 1} \rightarrow 1\ 1\ 0\ 0\ 0\ 1\ 1$ which means $.1100011_B \rightarrow 0.7734375_D$
- **Interval** $.1100011_B \rightarrow 0.7734375_D$ **compares** to $0.772352$ for **unlimited precision** and **lies** within $[0.7712, 0.77408)$ for **unlimited precision**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Arithmetic coding: Generating binary code



## Arithmetic decoding with binary code

- **Using incremental decoding** with **following questions**
  - (**1**) How to **start decoding**?
    - For **unambiguous decoding** → **number** of **bits** → **should point** to **smaller interval** than **smallest tag interval**
  - (**2**) How to **continue decoding**?
    - **Straightforward** procedure → **mimicking** incremental **encoder algorithm**
  - (**3**) How to **stop decoding**?
- **Example**
  - Considering **word length** $6$ and **received bits** $110001100 \ldots 0$, **alphabet** $\mathcal{A} = \{a_1, a_2, a_3\}$, $P(a_1) = 0.8$, $P(a_2) = 0.02$ and $P(a_3) = 0.18$
    - (**1**) **First** $6$ **bits corresponding** → **Tag** $0.765625$ → **first sequence element 1**
    - (**2**) **lower/upper** limits $l^{(n)} = l^{(n-1)} + \left(u^{(n-1)} - l^{(n-1)}\right)F_X(x_n - 1)$ and $u^{(n)} = l^{(n-1)} + \left(u^{(n-1)} - l^{(n-1)}\right)F_X(x_n)$ for first **sequence element 1**
      $l^{(1)} = 0 + (1 - 0)0 = 0$ and
      $u^{(1)} = 0 + (1 - 0)0.8 = 0.8$ → **Tag interval** $[0, 0.8)$ →**not confined** → **proceed**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Arithmetic coding: Generating binary code

## Arithmetic decoding with binary code

### Example

- Considering **word length** $6$ and **received bits** $110001100 \ldots 0$, **alphabet** $\mathcal{A} = \{a_1, a_2, a_3\}$, $P(a_1) = 0.8$, $P(a_2) = 0.02$ and $P(a_3) = 0.18$
  - **(2) lower/upper** limits $l^{(n)} = l^{(n-1)} + \left(u^{(n-1)} - l^{(n-1)}\right)F_X(x_n - 1)$ and $u^{(n)} = l^{(n-1)} + \left(u^{(n-1)} - l^{(n-1)}\right)F_X(x_n)$ for first **sequence element 1**
    $l^{(1)} = 0 + (1 - 0)0 = 0$ and
    $u^{(1)} = 0 + (1 - 0)0.8 = 0.8$ → **Tag interval** $[0, 0.8)$ → **not confined**→ **proceed**
  - **(3) Tag** $0.765625$ → **lies** in **top** $18\%$ of **interval** $[0, 0.8)$ → second **sequence element 3**
  - **(4) lower/upper** limits for second **sequence element 3**
    $l^{(2)} = 0 + (0.8 - 0)F_X(2) = 0.8 \cdot 0.82 = 0.656$ and
    $u^{(2)} = 0 + (0.8 - 0)F_X(3) = 0.8 \cdot 1.0 = 0.8$ → **Tag interval** $[0.656, 0.8)$ → **confined upper half** → **shift out 1** from **received buffer** → **moving** to **next bit** → **updating tag interval**
  - **(5)** $E_2: [0.5, 1) \rightarrow [0, 1)$ $\quad E_2(x) = 2(x - 0.5)$
    $l^{(2)} = 2 \cdot (0.656 - 0.5) = 0.312$ and $u^{(2)} = 2 \cdot (0.8 - 0.5) = 0.6$ → **Tag interval** $[0.312, 0.6)$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.



© Multimedia Technology Group

# Arithmetic coding: Generating binary code

## Arithmetic decoding with binary code

### Example

- Considering **word length** $6$ and **received bits** $110001100 \ldots 0$, **alphabet** $\mathcal{A} = \{a_1, a_2, a_3\}$, $P(a_1) = 0.8$, $P(a_2) = 0.02$ and $P(a_3) = 0.18$
  - **(5)** $E_2\colon [0.5, 1) \rightarrow [0, 1)$ $\quad E_2(x) = 2(x - 0.5)$
    $l^{(2)} = 2 \cdot (0.656 - 0.5) = 0.312$ and $u^{(2)} = 2 \cdot (0.8 - 0.5) = 0.6$ → **Tag interval** $[0.312, 0.6)$
  - **(6) Tag** $0.546875$ after **shifting bits**→ **lies** in **top** $80 - 82\%$ of **interval** $[0.312, 0.6)$→ third **sequence element 2**
  - **(7) lower/upper** limits $l^{(n)} = l^{(n-1)} + \left(u^{(n-1)} - l^{(n-1)}\right) F_X(x_n - 1)$ and $u^{(n)} = l^{(n-1)} + \left(u^{(n-1)} - l^{(n-1)}\right) F_X(x_n)$ for third **sequence element 2**
    $l^{(3)} = 0.312 + (0.6 - 0.312) F_X(1) = 0.312 + 0.288 \cdot 0.8 = 0.5424$ and $u^{(3)} = 0.312 + (0.6 - 0.312) F_X(2) = 0.312 + 0.288 \cdot 0.82 = 0.548168$ **Tag interval** $[0.5424, 0.548168)$ → **confined upper half** → **shift out 1** from **received buffer** → **moving** to **next bit** → **updating tag interval**
  - **(8)** $E_2\colon [0.5, 1) \rightarrow [0, 1)$ $\quad E_2(x) = 2(x - 0.5)$
    $l^{(3)} = 2 \cdot (0.5424 - 0.5) = 0.0848$ and $u^{(3)} = 2 \cdot (0.54816 - 0.5) = 0.09632$ → **Tag interval** $[0.0848, 0.09632)$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.



© Multimedia Technology Group

# Arithmetic coding: Generating binary code

## Arithmetic decoding with binary code

### Example

- Considering **word length** 6 and **received bits** $110001100 \ldots 0$, **alphabet** $\mathcal{A} = \{a_1, a_2, a_3\}$, $P(a_1) = 0.8$, $P(a_2) = 0.02$ and $P(a_3) = 0.18$
  - (**8**) $E_2$: $[0.5, 1) \rightarrow [0, 1)$ $\quad E_2(x) = 2(x - 0.5)$
    $l^{(3)} = 2 \cdot (0.5424 - 0.5) = 0.0848$ and $u^{(3)} = 2 \cdot (0.54816 - 0.5) = 0.09632$ → **Tag interval** $[0.0848, 0.09632)$ → **Tag** $000110$ → **rescaling**
  - (**9**) $E_1$: $[0, 0.5) \rightarrow [0, 1)$ $\quad E_1(x) = 2x$
    $l^{(3)} = 2 \cdot (0.0848) = 0.1696$ and $u^{(3)} = 2 \cdot (0.09632) = 0.19264$ → **Tag interval** $[0.1696, 0.19264)$ → **Tag** $001100$ → **rescaling**
  - (**10**) $E_1$: $[0, 0.5) \rightarrow [0, 1)$ $\quad E_1(x) = 2x$
    $l^{(3)} = 2 \cdot (0.1696) = 0.3392$ and $u^{(3)} = 2 \cdot (0.19264) = 0.38528$ → **Tag interval** $[0.3392, 0.38528)$ → **Tag** $011000$ → **rescaling**
  - (**11**) $E_1$: $[0, 0.5) \rightarrow [0, 1)$ $\quad E_1(x) = 2x$
    $l^{(3)} = 2 \cdot (0.3392) = 0.6784$ and $u^{(3)} = 2 \cdot (0.38528) = 0.77056$ → **Tag interval** $[0.6784, 0.77086)$ → **Tag** $110000$ → **rescaling**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.



© Multimedia Technology Group

# Arithmetic coding: Generating binary code



## Arithmetic decoding with binary code

### Example

- Considering **word length** $6$ and **received bits** $110001100 \ldots 0$, **alphabet** $\mathcal{A} = \{a_1, a_2, a_3\}$, $P(a_1) = 0.8$, $P(a_2) = 0.02$ and $P(a_3) = 0.18$
  - **(11)** $E_1: [0, 0.5) \rightarrow [0, 1)$ $\quad E_1(x) = 2x$
    $l^{(3)} = 2 \cdot (0.3392) = 0.6784$ and $u^{(3)} = 2 \cdot (0.38528) = 0.77056$ → **Tag interval** $[0.6784, 0.77086)$ → **Tag** $110000$ → **rescaling**
  - **(12)** $E_2: [0.5, 1) \rightarrow [0, 1)$ $\quad E_2(x) = 2(x - 0.5)$
    $l^{(3)} = 2 \cdot (0.6784 - 0.5) = 0.3568$ and $u^{(3)} = 2 \cdot (0.77056 - 0.5) = 0.54112$ → **tag interval** $[0.3568, 0.54112)$ → **tag** $100000$
  - **(13) Tag** $100000_B \rightarrow 0.5_D$ **lies** in $80\%$ of **interval** → **final fourth symbol 1**
- **Received bits** $110001100$ **translates** to received **decoded sequence** $\mathbf{1\ 3\ 2\ 1}$

### Mapping diminishing tag straddling around midpoint $[0.25, 0.75)$

- $E_3: [0.25, 0.75) \rightarrow [0, 1)$ $\quad E_3(x) = 2(x - 0.25)$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Arithmetic coding: Generating binary code



## Integer implementation of arithmetic coding

- Using **integer arithmetic** to **generate binary sequence**
  - Previously **described floating-point implementation**
  - For **word length** $m$ → **mapping values** $[0, 1)$ to **range** of $2^m$ **binary words**
    - **Point** $0$ **mapped** to $\underbrace{00\ldots0}_{m\text{ times}}$, $1$ **mapped** to $\underbrace{11\ldots1}_{m\text{ times}}$, and $0.5$ **mapped** to $1\underbrace{00\ldots0}_{m-1\text{ times}}$
  - **Replacing** CDF **floating-point** $F_X(x)$
    - **Using** $n_j$ as **number** of **times** symbol $j$ **occurs** in **sequence** of **length** $TotCount$
    - **Estimation** of $F_X(x) = \frac{\sum_{i=1}^{k} n_i}{TotCount}$ and **defining** $CumCount(k) = \sum_{i=1}^{k} n_i$
  - **Values** for **lower** $l^{(n)}$ and **upper** $u^{(n)}$ interval limits
    - $n$th **symbol** $x_n$ to be **encoded** and $\lfloor x \rfloor$ for **largest integer less than** or **equal** $x$

$$l^{(n)} = l^{(n-1)} + \left\lfloor \frac{\left(u^{(n-1)} - l^{(n-1)} + 1\right) CumCount(x_n - 1)}{TotCount} \right\rfloor \text{ and}$$

$$u^{(n)} = l^{(n-1)} + \left\lfloor \frac{\left(u^{(n-1)} - l^{(n-1)} + 1\right) CumCount(x_n)}{TotCount} \right\rfloor - 1$$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Arithmetic coding: Generating binary code



## Integer implementation of arithmetic coding

- **Word length $m$ → has** to be **large enough**
  - To **accommodate lower/upper limits** and **subintervals**
  - It **means** that $2^m > TotCount$ **or** $m > \lceil \log_2(TotCount) \rceil$
  - More **precisely** $\frac{1}{4}2^m > TotCount$ **or** $m > 2 + \lceil \log_2(TotCount) \rceil$
- **Encoder algorithm**

Initialize $l$ and $u$.
Get symbol.

$$l \leftarrow l + \left\lfloor \frac{(u - l + 1) \times Cum_Count(x-1)}{Total_Count} \right\rfloor$$

$$u \leftarrow l + \left\lfloor \frac{(u - l + 1) \times Cum_Count(x)}{Total_Count} \right\rfloor - 1$$

```
while (MSB of u and l are both equal to b or E3 condition holds)
if (MSB of u and l are both equal to b)
  {
    send b
    shift l to the left by 1 bit and shift 0 into LSB
    shift u to the left by 1 bit and shift 1 into LSB
    while(Scale3 > 0)
      {
        send complement of b
        decrement Scale3
        }
  }
if (E3 condition holds)
    {
      shift l to the left by 1 bit and shift 0 into LSB
      shift u to the left by 1 bit and shift 1 into LSB
      complement (new) MSB of l and u
      increment Scale3
      }
```

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Adaptive arithmetic coding



- **Construction** of **arithmetic coders**
  - Normally **distribution** of **source** (cumulative counts) is **available**
  - For **one-pass encoding**
    - **Various applications** → **counts not available a priory**
  - **Adaptive arithmetic coding needed**
    - **Coder modification** → **coder learns distribution** while **coding progresses**
      - **Starting** with **count 1** for **each letter alphabet**
      - **Initial counts** can be **established** to **reflect some knowledge**
      - **Counts** for **each letter incremented after** letter encoding
      - **Exact same updates** on **decoder side**
    - Necessary **picking word length independent** of **total count**
      - **Given word length** $m$ → only **total counts** $2^{m-2}$ **can** be **accommodated**
      - When **total count reaches** $2^{m-2}$ **rescaling/renormalization** applied
      - Simple **rescaling** → **dividing** all **counts** by 2 and **round up** (**no zero counts**)

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Binary arithmetic coding

- **Various applications** with **binary alphabet**
  - **Example**
    - **Binary source** putting **symbols** $P(0) = 0.125$ and $P(1) = 0.875$ with **probabilities**
      - For **directly encoding symbols** as $0s$ and $1s$ rate $1\ \mathrm{bit/symbol}$
      - Source **entropy** $H = 0.543\ \mathrm{bit/symbol}$
      - **Encoding** with **almost twice rate needed** and **redundancy** of $0.457\ \mathrm{bit/symbol}$
  - **Solution → arithmetic coding → popularity** for **encoding binary sources** with **skewed probabilities**
  - **Binary source**
    - **Binary** by its **nature**
      - **Bilevel** documents/images, …
    - **Binary representation** of **nonbinary data**
      - Context Adaptive Binary Arithmetic Coder (**CABAC**) in **H.264 video coding**
  - Simple **probability model** needed (**single number**)

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.



© Multimedia Technology Group

# Binary arithmetic coding

## General view of binary arithmetic coding

- **Normally keeping upper** $u^{(n)}$ and $l^{(n)}$ **lower interval limits**
  - **Binary coders** use **lower end** $l^{(n)}$ and **size** $A^{(n)}$ of **interval**
    - **Size** of **interval** $A^{(n)} = u^{(n)} - l^{(n)}$
    - **Tag** of $l^{(n)}$ → **binary representation** of $l^{(n)}$
    - **Update equation** for $A^{(n)}$ and $l^{(n)}$
      $A^{(n)} = A^{(n-1)}\left(F_X(x_n) - F_X(x_n - 1)\right) = A^{(n-1)}P(x_n)$ and
      $l^{(n)} = l^{(n-1)} + A^{(n-1)}F_X(x_n - 1)$
  - **Mapping** of **symbols** into
    - **MPS** (More Probable Symbol) and **LPS** (Less Probable Symbol)
    - **Probability** of **LPS occurrence** denoted as $q_c$
    - **Update equation** for $A^{(n)}$ and $l^{(n)}$ for **MPS occurrence**
      $l^{(n)} = l^{(n-1)}$ and
      $A^{(n)} = A^{(n-1)}(1 - q_c)$
    - **Update equation** for $A^{(n)}$ and $l^{(n)}$ for **LPS occurrence**
      $l^{(n)} = l^{(n-1)} + A^{(n-1)}(1 - q_c)$ and
      $A^{(n)} = A^{(n-1)}q_c$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.



© Multimedia Technology Group

# Binary arithmetic coding: QM coder



## QM coder computationally efficient implementation

- **Used** in **JBIG** (Joint Bi-level Image Experts Group)
  - **JBIG** algorithm for bi-level **images compression**
  - **Multiplications** (**HW/SW computationally expensive**) are **avoided**
    - **Assuming** $A^{(n)}$ has **value close** to **one** → **approximation** of **multiplications** by 1
  - **Mapping** of **symbols** into
    - **MPS** (More Probable Symbol) and **LPS** (Less Probable Symbol)
    - **Probability** of **LPS occurrence** denoted as $q_c$
    - **Update equation** for $A^{(n)}$ and $l^{(n)}$ for **MPS occurrence**
      $l^{(n)} = l^{(n-1)}$ and
      $A^{(n)} = 1 - q_c$
    - **Update equation** for $A^{(n)}$ and $l^{(n)}$ for **LPS occurrence**
      $l^{(n)} = l^{(n-1)} + (1 - q_c)$ and
      $A^{(n)} = q_c$
  - For $A^{(n)}$ **below** 0.75 → **rescaling** as **repeated doubling** (left/right **shifts**)
  - **LPS occurrence** denoted as $q_c$ is **updated with rescaling**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Binary arithmetic coding: M coder



## M (modulo) computationally efficient implementation

- **Another variant** of **QM coder** used in **CABAC** of **H.264**
  - **Multiplications** are **avoided** → **replaced** with **table lookup**
    - **Update equation** for $A^{(n)}$ and $l^{(n)}$ for **MPS occurrence**
      $l^{(n)} = l^{(n-1)}$ and
      $A^{(n)} = A^{(n-1)}(1 - q_c)$
    - **Update equation** for $A^{(n)}$ and $l^{(n)}$ for **LPS occurrence**
      $l^{(n)} = l^{(n-1)} + A^{(n-1)}(1 - q_c)$ and
      $A^{(n)} = A^{(n-1)} q_c$
  - **Only multiplications** between **range** $A^{(n-1)}$ and **LPS** $q_c$ **probability**
    - **M coder probability** to **take specific values** → **replaced** with **table lookup**
    - **LPS probability** $q_c$ takes only **64 possible** $q_m$ **values**
      $q_m = \alpha q_{m-1}, m = 1, 2, \dots, 63$
      $\alpha = \left(\frac{0.01875}{0.5}\right)^{\frac{1}{63}}$ and $q_0 = 0.5$
    - **Range restricted** to **four values**
      $A = A_{min} + (k+1)\Delta A, k = 0, 1, 2, 3$, and $\Delta A = (A_{max} - A_{min})/4$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Huffman and Arithmetic coding



## Comparison of Huffman and Arithmetic

- **Arithmetic coder** more **complicated** than **Huffman**
  - **Performance depends** on how **scheme** is **used**
- **Example**
  - Consider **four letter alphabet** $\mathcal{A} = \{a_1, a_2, a_3, a_4\}$ with **probabilities** $P(a_1) = 1/2$, $P(a_2) = 1/4$, $P(a_3) = 1/8$ and $P(a_3) = 1/8$

| Symbol | $F_X$ | $\overline{T}_X$ | In Binary | $\lceil \log \frac{1}{P(x)} \rceil + 1$ | Code |
|---|---|---|---|---|---|
| 1 | .500 | .2500 | .0100 | 2 | 01 |
| 2 | .750 | .6250 | .1010 | 3 | 101 |
| 3 | .875 | .8125 | .1101 | 4 | 1101 |
| 4 | 1.000 | .9375 | .1111 | 4 | 1111 |

- **Entropy (Huffman)** $H = 1.75$ bits/symbol
- **Average code length** for **one symbol** sequences $\bar{l} = 2.75$ bits/symbol
- For **two symbol** sequences $\bar{l} = 2.25$ bits/symbol

| Message | $P(x)$ | $\overline{T}_X(x)$ | $\overline{T}_X(x)$ in Binary | $\lceil \log \frac{1}{P(x)} \rceil + 1$ | Code |
|---|---|---|---|---|---|
| 11 | .25 | .125 | .001 | 3 | 001 |
| 12 | .125 | .3125 | .0101 | 4 | 0101 |
| 13 | .0625 | .40625 | .01101 | 5 | 01101 |
| 14 | .0625 | .46875 | .01111 | 5 | 01111 |
| 21 | .125 | .5625 | .1001 | 4 | 1001 |
| 22 | .0625 | .65625 | .10101 | 5 | 10101 |
| 23 | .03125 | .703125 | .101101 | 6 | 101101 |
| 24 | .03125 | .734375 | .101111 | 6 | 101111 |
| 31 | .0625 | .78125 | .11001 | 5 | 11001 |
| 32 | .03125 | .828125 | .110101 | 6 | 110101 |
| 33 | .015625 | .8515625 | .1101101 | 7 | 1101101 |
| 34 | .015625 | .8671875 | .1101111 | 7 | 1101111 |
| 41 | .0625 | .90625 | .11101 | 5 | 11101 |
| 42 | .03125 | .953125 | .111101 | 6 | 111101 |
| 43 | .015625 | .9765625 | .1111101 | 7 | 1111101 |
| 44 | .015625 | .984375 | .1111111 | 7 | 1111111 |

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

# Huffman and Arithmetic coding



## Comparison of Huffman and Arithmetic

- **Arithmetic coder** more **complicated** than **Huffman**
  - **Performance depends** on how **scheme** is **used**
  - **Average length** of **arithmetic code** $\bar{l}_A$ for **sequence** of **length** $m$
    - It **can be shown** → **average code length** $\bar{l}_A$ → $H(X) \leq \bar{l}_A < H(X) + \frac{2}{m}$
    - **Increasing sequence length** $m$ → $\bar{l}_A$ as **close** to **entropy** $H(X)$ as **desired**
  - **Huffman code** for **block symbols** $m$
    - It **can be shown** → **average code length** $\bar{l}_H$ → $H(X) \leq \bar{l}_H < H(X) + \frac{1}{m}$
  - **Huffman procedure requires building entire code** → **codebook size** $k^m$
    - For **original alphabet** $k = 16$ and **blocks** $m = 20$ → **codebook size** $16^{20}$
    - **Increasing block** size → **not viable** (**huge Huffman codebook**)
    - **Optimal only** for **probabilities** in **powers** of **two**
  - **Arithmetic coding** → **easier** to **implement** and **more efficient**

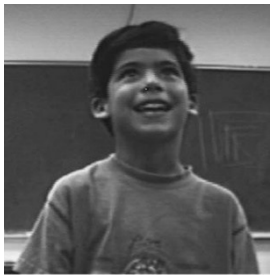
Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

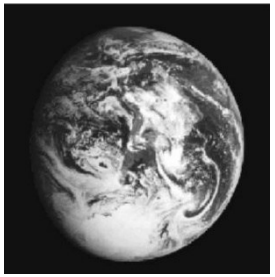
# Applications

## Simple lossless image compression

### Example compression for 8bpp images

- Intensity **values** 0..255
- **Images** of 256 × 256 **pixels**
- → resulting in 65 536 **bytes uncompressed**





**Compression using Huffman and adaptive ARITHMETIC codes on pixel values**

| Bits/ Pixel | Total Size (bytes) | Compression Ratio (arithmetic) | Compression Ratio (Huffman) |
|---|---|---|---|
| 6.52 | 53,431 | 1.23 | 1.16 |
| 7.12 | 58,306 | 1.12 | 1.27 |
| 4.67 | 38,248 | 1.71 | 1.67 |
| 6.84 | 56,061 | 1.17 | 1.14 |

**Compression using Huffman and adaptive ARITHMETIC codes on pixel DIFFERENCE values**

| Bits/ Pixel | Total Size (bytes) | Compression Ratio (arithmetic) | Compression Ratio (Huffman) |
|---|---|---|---|
| 3.89 | 31,847 | 2.06 | 2.08 |
| 4.56 | 37,387 | 1.75 | 1.73 |
| 3.92 | 32,137 | 2.04 | 2.04 |
| 6.27 | 51,393 | 1.28 | 1.26 |

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.



© Multimedia Technology Group

# References and recommended materials


- [1] Sayood, K.: **Introduction to Data Compression**, Morgan Kaufmann Publishers, 2006.
- [2] Sayood K.: **Lossless Compression Handbook**, Elsevier, 2003.
- [3] Kahrs, M., Brandenburg, K.: **Applications of Digital Signal Processing to Audio and Acoustics**, Kluwer Academic Publishers, 1998.
- [4] Gonzalez, R. C., Woods, R. E.: **Digital Image Processing**, Pearson, 2007.
- [5] Salomon, D., Motta, G.: **Handbook of Data Compression**, Springer-Verlag London, 2010.
- [6] Salomon, D.: **A Concise Introduction to Data Compression**, Springer, 2008.
- [7] Thyagarajan, K. S.: **Still Image and Video Compression with MATLAB**, Willey, 2011.




© Multimedia Technology Group

Thank you for your attention!

MultiMedia Technology Group

Czech Technical University in Prague
Faculty of Electrical Engineering
Department of Radioelectronics

Technická 2
166 27 Prague 6
Czech Republic



© Multimedia Technology Group